

Clinical Guidelines Q&A Assistant

A Retrieval-Augmented Generation (RAG) System for Evidence-Based Clinical Lookup

[Your Name] | [Course/Program] | [Date]

2. Business Details about the Project

Clinicians, medical students, and healthcare learners often need to quickly find accurate treatment information from lengthy clinical guideline documents such as ADA, ACC/AHA, GINA, and GOLD. Manually searching through 100+ page PDFs is time-consuming and increases the risk of missing key information. This project solves that problem by building a Retrieval-Augmented Generation (RAG) system that retrieves the exact relevant sections of a guideline document and generates a structured, cited answer — instead of relying on an AI model's memory, which can produce incorrect or outdated medical information.

This is a single-user educational tool built for learning purposes and is not intended for real clinical deployment or decision-making.

Core Functional Areas:

Function	Description
PDF Ingestion	Loads raw guideline PDFs from the data folder
Text Chunking	Splits documents into smaller overlapping segments
Embedding Generation	Converts text chunks into vector representations using MiniLM
Vector Storage	Stores embeddings in a FAISS vector index
Similarity Retrieval	Fetches the most relevant chunks for a given question
Prompt Construction	Combines retrieved context and user question into a structured prompt
Answer Generation	Uses an LLM to generate an evidence-backed, structured response

Function	Description
Citation Formatting	Maps each answer back to its source file name and page number
UI Interaction	Provides a chat-based interface using Streamlit
Clinical Safety Disclaimer	Appends a clinical judgment warning to every generated answer

3. Technical Details about the Project

This project follows a simple Retrieval-Augmented Generation (RAG) architecture. Guideline PDFs are first processed offline into a searchable vector index. At query time, the system retrieves relevant chunks of guideline text and passes them to a language model, which generates an answer strictly grounded in that retrieved content.

Architecture Flow:

```
PDF Files → Text Splitter → Embedding  
Model (MiniLM) → FAISS Index
```

|

User Question → Retriever → Relevant
Chunks → Prompt Template → ChatOpenAI
(GPT-4o) → Formatted Answer → Streamlit UI

Data Flow Steps:

1. User submits a question through the Streamlit chat input.
2. The retriever embeds the question and searches the FAISS index for the top matching chunks.
3. Retrieved chunks are formatted into a context string along with their source citations.
4. The context and question are inserted into a structured system prompt.
5. The prompt is sent to ChatOpenAI (GPT-4o), which generates a 4-part structured answer.
6. The answer, source citations, and clinical disclaimer are displayed in the UI.

Technical Considerations:

Consideration	Details
Chunk Size & Overlap	1000 characters with 150 character overlap to preserve context across chunk boundaries
Local Embeddings	MiniLM runs on CPU locally, avoiding per-query API costs for retrieval

Consideration	Details
Temperature Setting	Set to 0 for consistent, deterministic answers
Caching	RAG chain cached using Streamlit's cache_resource to avoid reloading on every question
Index Deserialization	allow_dangerous_deserialization is set to True since the FAISS index is self-generated and trusted

Environment / Infrastructure:

Item	Detail
Language	Python 3.x
Execution	Fully local, except for the OpenAI API call for answer generation
Vector Store	FAISS index stored locally under vectorstore/faiss_index/
Environment Variables	OPENAI_API_KEY stored in .env file

 **[Paste screenshot of your vectorstore/faiss_index/ folder showing index.faiss**

and index.pkl files here]

4. Implementation

This section walks through how each part of the pipeline was implemented, with the actual code used and the corresponding output.

4.1 PDF Ingestion, Chunking, and Embedding

Guideline PDFs placed in `data/raw/` are loaded, split into overlapping chunks, embedded using a local MiniLM model, and saved as a FAISS index. This is a one-time (or re-run-when-needed) step handled by `src/ingest.py`.

```
def load_documents():
    loader =
PyPDFDirectoryLoader(RAW_DATA_DIR)
    documents = loader.load()
    return documents

def split_documents(documents):
    splitter =
RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        separators=["\n\n", "\n", ". ", "
", ""],
    )
```

```

        chunks =
        splitter.split_documents(documents)
        return chunks

def build_and_save_index(chunks):
    embeddings = HuggingFaceEmbeddings(
        model_name=EMBEDDING_MODEL_NAME,
        model_kwargs={"device": "cpu"},
    )
    vectorstore =
    FAISS.from_documents(chunks, embeddings)
    vectorstore.save_local(FAISS_INDEX_DIR)

```

This was run using:

```
python -m src.ingest
```

 [Paste screenshot of terminal output showing number of pages loaded, number of chunks created, and "FAISS index saved" message]

4.2 Loading the Index and Configuring the Retriever

Once the FAISS index is built, `src/retriever.py` reloads it using the same embedding model and exposes it as a retriever that returns the top-k most relevant chunks for any question.

```

def load_vectorstore():
    embeddings = HuggingFaceEmbeddings(

```

```

        model_name=EMBEDDING_MODEL_NAME,
        model_kwargs={"device": "cpu"},
    )
    vectorstore = FAISS.load_local(
        FAISS_INDEX_DIR,
        embeddings,

allow_dangerous_deserialization=True,
    )
    return vectorstore

def get_retriever(top_k: int = 4):
    vectorstore = load_vectorstore()
    retriever = vectorstore.as_retriever(
        search_type="similarity",
        search_kwargs={"k": top_k},
    )
    return retriever

```

Tested independently using:

```

from src.retriever import get_retriever

retriever = get_retriever(top_k=4)
docs = retriever.invoke("What is the first-
line treatment for hypertension?")
for doc in docs:
    print(doc.metadata,
          doc.page_content[:200])

```

 **[Paste screenshot of retrieved chunk metadata and text from your terminal test]**

4.3 Prompt Construction and Answer Generation

`src/rag_chain.py` combines the retrieved chunks, a structured system prompt, and ChatOpenAI into a single runnable chain. The prompt enforces the required 4-section output format.

```
SYSTEM_PROMPT = """You are a Clinical
Guidelines Q&A Assistant.
Answer the user's question using ONLY the
CONTEXT below...

### 1. Evidence-Backed Answer
### 2. Treatment Pathway Summary
### 3. Guideline Citation
### 4. Clinical Judgment Warning

CONTEXT:
{context}
"""

def build_rag_chain(top_k: int = 4):
    retriever = get_retriever(top_k=top_k)
    llm = ChatOpenAI(model=LLM_MODEL_NAME,
temperature=0)

    chain = (
        {
            "context": retriever |
_format_docs_for_prompt,
            "question":
RunnablePassthrough(),
```

```

    }
    | prompt
    | llm
    | StrOutputParser()
)
return chain, retriever

```

Tested independently using:

```

from src.rag_chain import ask_question

answer, docs, chunks_text =
ask_question("What is the first-line
treatment for hypertension?")
print(answer)

```

 **[Paste screenshot of terminal output showing the full 4-section generated answer]**

4.4 Citation Formatting

`src/utils.py` extracts the file name and page number from each retrieved chunk's metadata so the answer can reference exact sources.

```

def format_citation(doc) -> str:
    source_path =
doc.metadata.get("source", "Unknown
source")
    file_name =
os.path.basename(source_path)
    page = doc.metadata.get("page")

```

```

    if page is not None:
        return f"{file_name} (page {page + 1})"
    return file_name

```

📸 [Optional: paste screenshot showing a citation line, e.g., "GINA_2024.pdf (page 13)"]

4.5 Streamlit UI

`ui/app.py` ties everything together into a chat interface: it caches the RAG chain, takes the user's question, displays it, shows a loading spinner, then renders the structured answer, disclaimer, and an expandable panel of retrieved source chunks.

```

question = st.chat_input("Ask a clinical guideline question...")

if question:
    st.markdown(f"**👤 You asked:** {question}")
    st.divider()

    with st.spinner("Retrieving guidelines and generating answer..."):
        answer = chain.invoke(question)
        retrieved_docs = retriever.invoke(question)

    st.markdown("## Answer")
    st.markdown(answer)

```

```

st.markdown(get_disclaimer())

with st.expander("📄 View retrieved
source chunks"):
    for i, doc in
enumerate(retrieved_docs, start=1):
        st.markdown(f"***[{i}]
{format_citation(doc)}***")

st.text(doc.page_content.strip())

```

Run using:

```
streamlit run ui/app.py
```

📸 [Paste screenshot of the Streamlit app running in the browser, showing the question, generated answer, and source chunks expander]

5. Sample Input and Output

Example 1 — Standard Guideline Question

Question: [Type the exact question you asked, e.g., "What is the first-line treatment for stage 1 hypertension?"]

📸 [Paste screenshot of the full answer output including all 4 sections]

📸 [Paste screenshot of the expanded source chunks panel showing citations]

Example 2 – Alternative/Second-Line Therapy

Question

Question: [Type your second example question here]

 [Paste screenshot of the answer output]

 [Paste screenshot of the source citations]

Example 3 – Question With No Matching Guideline

Content

Question: [Type a question outside your ingested PDFs' scope]

 [Paste screenshot showing the "insufficient information" style response]

6. Tech Stack

Component	Purpose
Python	Core programming language
LangChain	Framework for RAG orchestration
langchain-openai	Integration with ChatOpenAI

Component	Purpose
langchain-community	Loaders, vector store, and embedding wrappers
ChatOpenAI (GPT-4o)	Large language model for answer generation
sentence-transformers (all-MiniLM-L6-v2)	Local embedding model
FAISS (faiss-cpu)	Vector similarity search and storage
PyPDFDirectoryLoader	Loads PDF files for ingestion
RecursiveCharacterTextSplitter	Splits documents into chunks
Streamlit	Frontend chat interface
python-dotenv	Loads environment

Component	Purpose
	variables from .env file

No containerization was used in this project — it runs locally through a Python virtual environment.

7. One Best Benefit for Business

This project centralizes scattered clinical guideline knowledge into a single, citation-backed lookup tool, significantly reducing the time spent manually searching through lengthy PDF documents while maintaining traceability back to the original source.

Key Impacts:

- Faster lookup of first-line versus alternative treatment options
- Built-in citation trail (file name and page number) supports easy verification
- Consistent, structured 4-part answer format improves usability for learners
- Local embedding generation reduces cost compared to using an API for every search
- Reduces dependence on memorized or outdated guideline knowledge

- Easily extensible to additional guideline sets by adding new PDFs and re-running ingestion